EX02 Report: Shared State

Student Name: Maurits van Staalduinen
Student ID: `c9881018`

Claude Sonnet 5.5 used for handin formating, table formating, and clearer wording

Task 1 - Replicated Counter

1.1 - Max-Based Synchronization

replicated_max.py, one program started per node with its name, port and the peer address

nodes talk over UDP, a receive thread merges incoming values with max, the main thread increments (counter behind a lock)

after every increment the node sends its counter, after the increments it keeps sending every 0.5s for 10 rounds so a lost message gets resent

```
with lock:
    counter = max(counter, received)
```

A does 10 increments, B 7

```
ssh ec2-user@52.70.228.101 'python3 replicated_max.py A 5000
172.31.94.103:5001 10'
ssh ec2-user@54.152.240.223 'python3 replicated_max.py B 5001
172.31.91.207:5000 7'
```

```
[A 19:09:45] increments stopped (10 local), only syncing now
[A 19:09:50] final counter = 17
[B 19:09:45] recv 17, counter 16 -> 17
[B 19:09:50] final counter = 17
```

both end with 17, max only goes up and the order of merges doesn't matter, so once both saw the highest value they stay equal

17 is only correct here because every message arrived (<1 ms) before the next local increment, the increments never overlapped

1.2 - Concurrent Increments

result is 1 on both nodes, should be 2: A has 1, B has 1, max(1, 1) = 1

added an optional recv delay argument (receive thread starts after n seconds), so both increment before merging anything

```
[A 13:56:58] inc, counter = 1
[A 13:57:00] recv 1, counter stays 1
[A 13:57:03] final counter = 1
```

B looks the same, 1 increment lost

the counter doesn't say who incremented, so "B did its own increment" and "B already has my increment" look the same to max

agreeing only means the replicas converged, not that the state contains all updates, the lost increment isn't visible anywhere

1.3 - Message Loss, Duplication and Reordering

SITUATION	PROBLEM?	WHY
lost	no, only delayed	every message has the whole counter, the next one fixes it
duplicate	no	max(c, x) twice = once
older after newer	no	smaller value, max ignores it

it would be a problem if "+1" was sent instead of the counter, the real problem is concurrent increments (1.2), not the network

1.4 - Preventing Lost Increments

replicated_gcounter.py, same structure, but each node keeps one count per node, e.g. {"A": 10, "B": 7}, and only increments its own entry

the whole map is sent as JSON, merged per entry with max, the value is the sum

```
for n, c in received.items():
    counts[n] = max(counts.get(n, 0), c)
```

every entry has only one writer, so its max is always that node's newest count, nothing gets lost, and lost / duplicate / reordered messages are still fine

run with recv delay, 1 increment each (situation of 1.2) and 10 + 7

RUN	REPLICATED_MAX.PY	REPLICATED_GCOUNTER.PY
A 1, B 1	1	2
A 10, B 7	10	17

```
[A 15:56:57] final counts = {'A': 10, 'B': 7}, value = 17
[B 15:56:56] final counts = {'B': 7, 'A': 10}, value = 17
```

Task 2 - Single-Copy Counter

Chosen data store: Redis

2.1 - Safe Increment

redis 6.2 on a third instance (installed in user_data of main.tf, port 6379 only open inside the security group), nodes only talk to redis

single_copy.py, mode argument picks the increment (unsafe, incr, lock, watch), both nodes INCR a "ready" key and wait until it is 2 so they really start together

```
def incr_atomic():
    return r.incr("counter"), 0
```

1000 increments per node

```
[A 16:02:33] done, 1000 increments in 0.284s (0.284 ms per
increment)
[A 16:02:33] counter now = 1855
[B 16:02:33] counter now = 2000
```

2000, nothing lost, A saw 1855 when done so the increments overlapped

no sync messages needed: only one copy exists, redis runs commands one after the other, each INCR sees the one before it no matter which node sent it

2.2 - Simultaneous Increments

read-modify-write can interleave: A reads 5, B reads 5, both write 6, one increment lost

```python
def incr_unsafe():
    value = int(r.get("counter"))
    time.sleep(hold)
    r.set("counter", value + 1)
```

1000 each gave 1147 instead of 2000, with a 1s hold between GET and SET 3 each gave 3 instead of 6

redis prevents it because INCR is one command and redis executes one command at a time, so nothing can run between the read and the write

besides INCR I tried a lock (SET lock NX PX 10000, retry every 1ms) and WATCH + MULTI/EXEC (EXEC fails if counter changed, retry), both give 2000

second update while another is in progress (hold 1s, 3 increments per node):

MODE	SECOND UPDATE	RESULT
unsafe	completes, overwrites	3 (wrong)
incr	completes right after the other command	6
lock	waits for the lock (B waited 3s, lock isn't fair)	6
watch	EXEC fails, retried with new value (B 3 retries)	6

Task 3 - Comparison

3.1 - Measurements (100 Increments)

timed with time.perf_counter() inside the programs, replicated with bench_replicated.py (gcounter without sleeps, 5 runs), single-copy with single_copy.py (3 runs, alone = only A, concurrent = A and B)

IMPLEMENTATION	MEASUREMENT	100 INCREMENTS	PER INCREMENT
Replicated	local update	0.16 ms	1.6 us
Replicated	sending state (UDP)	2.6 ms	26 us
Replicated	sync until equal	1.9 ms	-
Single-copy	INCR alone / concurrent	27.7 / 26.3 ms	0.28 / 0.26 ms
Single-copy	lock alone / concurrent	107 / 203 ms	1.07 / 2.03 ms
Single-copy	watch alone / concurrent	78 / 133 ms	0.78 / 1.33 ms

value was correct in every run

replicated increments are a local memory write, sending doesn't wait for an answer, the replicas are equal ~2 ms after the last increment

single-copy needs a round trip to redis per increment, but the value is in the store right away

INCR doesn't get slower with concurrent updates (redis runs a command in microseconds), lock and watch need 3-4 commands and get about 2x slower because nodes wait for the lock or redo failed transactions

3.2 - Discussion Questions

> *Question 1: Where does synchronization happen in each implementation?*

replicated: between the nodes, asynchronously in the receive thread (merge)

single-copy: inside redis, which runs one command at a time / decides who gets the lock or whose transaction goes through

> *Question 2: What happens if the two nodes cannot communicate temporarily?*

replicated: both keep incrementing locally but drift apart, after reconnecting the per-node counts merge to the correct total

single-copy: nothing, the nodes only talk to redis

> *Question 3: What happens if the shared data store is unavailable?*

single-copy: increments fail with a connection error, redis is a single point of failure (a held lock still expires after 10s)

replicated: not affected, there is no store

> *Question 4: Which approach would you choose if increments must be immediately visible to both nodes? Why?*

single-copy with INCR, once it returns the value is in redis and every read sees it, the replicated counter is only eventually visible (and not at all during a partition)

cost is a round trip per increment and no increments while redis is down